

Reporte Tarea 7

Jesús Galaviz Tapia

En este trabajo se resolvió numéricamente el problema de Thomson usando Python: se implementaron los métodos de **random walk** y el de **descenso de gradiente** propuesto por Claxton y Benson con el fin de calcular la energía electrostática (en unidades de $1/4\pi\epsilon_0$) y las configuraciones correspondientes de N partículas con carga $+1$ distribuidas sobre una esfera unitaria. Dicha energía está dada por

$$U_T = \frac{1}{2} \sum_{i=1}^N \sum_{\substack{j=1 \\ j \neq i}}^N \frac{1}{r_{ij}},$$

donde r_{ij} es la distancia euclidiana entre las partículas i y j .

El archivo de salida especifica las configuraciones de las partículas obtenidas y su correspondiente energía electrostática, en formato **x,y,z**.

En este trabajo se emplearon las librerías NumPy y Numba. La librería Numba se utilizó para compilar el código a un backend en C, evitando así la ejecución mediante el intérprete de Python y por lo tanto disminuyendo el tiempo de cómputo. Numba sólo puede optimizar funciones que operan con estructuras y métodos de NumPy (por ejemplo al definir la matriz que contiene las posiciones de las partículas o al generar un número aleatorio), por ello fué indispensable el uso de esta última.

Parámetros

A lo largo de la simulación se usaron los siguientes parámetros, posteriormente se presentan los métodos usados. Se indica en qué métodos se usó cada parámetro.

- N : número de partículas puntuales. (métodos 1,2,3)
- m : número de ciclos Montecarlo. (método 1)
- w : número total de iteraciones del algoritmo Claxton–Benson. (métodos 2,3)
- K : parámetro de control que regula la magnitud del desplazamiento de las partículas en cada iteración. En este caso, se usó $K = 0.15$. (métodos 2,3)

Random Walk (método 1)

Metodología

El método empleado corresponde a una implementación del algoritmo de *random walk* mediante el enfoque de Monte Carlo para minimizar la energía electrostática de un conjunto de partículas puntuales sobre una esfera.

1. **Inicialización de coordenadas.** Se genera un conjunto inicial de N posiciones aleatorias sobre la superficie de la esfera unitaria. Para cada partícula i , se eligen tres coordenadas aleatorias (x_1, x_2, x_3) en el intervalo $[-1, 1]$ y se normaliza el vector resultante:

$$\vec{r}_i = \frac{(x_1, x_2, x_3)}{\sqrt{x_1^2 + x_2^2 + x_3^2}},$$

asegurando que todas las partículas se ubiquen en la superficie de la esfera de radio unitario.

2. **Cálculo de energía.** La energía total del sistema se calcula mediante la función `f_u(crd)`, que realiza una doble suma sobre todos los pares (i, j) , evaluando la interacción electrostática U_T .

Para calcular la energía parcial asociada a una sola partícula k , se utiliza la función `f_eng(x1, x2, x3, k, crd)`, que evalúa la contribución de dicha partícula con todas las demás. Es decir, calcula

$$U = \sum_{\substack{i=1 \\ i \neq k}}^N \frac{1}{r_{i,k}}.$$

3. **Evolución Monte Carlo.** En cada ciclo de Monte Carlo, implementado en la función `MCS(crd, N, m)`, se realizan los siguientes pasos:

- Se selecciona una partícula al azar.
- Se genera una posición *trial* desplazando aleatoriamente sus coordenadas.
- La posición *trial* se normaliza nuevamente para mantener la partícula sobre la esfera.
- Se evalúa la energía electrostática de la partícula en la posición propuesta y en la posición actual. Si

$$U_{\text{trial}} - U_{\text{actual}} = \Delta U < 0,$$

la nueva posición es aceptada; en caso contrario, se rechaza el movimiento. Esto es equivalente a que $U_T' < U_T$, es decir, que la energía total del sistema cuando cambiamos la partícula k por la posición *trial* es menor que la correspondiente a la posición anterior. Esto se demuestra en el apéndice.

4. **Obtención de resultados.** Tras completar los m ciclos, se calcula la energía total del sistema mediante `f_u(crd)`. Las coordenadas finales se guardan en el archivo de texto `coord_R_N.txt` (para N el número de partículas) junto con el valor final de la energía electrostática. Finalmente, se compara el resultado obtenido con el valor exacto correspondiente a N para determinar el error porcentual.

Claxton-Benson (método 2)

Metodología

El principio fundamental del método consiste en mover las partículas en la dirección de la fuerza electrostática resultante que actúa sobre cada una, pero con una magnitud controlada por un factor adaptativo γ , de modo que todas permanezcan sobre la superficie de la esfera. Contiene los siguientes pasos:

1. **Inicialización de las posiciones.** Se generan N posiciones aleatorias sobre la esfera unitaria utilizando la función `f_coord(N)`. Cada partícula se obtiene normalizando un vector aleatorio tridimensional (x_1, x_2, x_3) cuyos componentes se distribuyen uniformemente en $[-1, 1]$:

$$\vec{r}_i = \frac{(x_1, x_2, x_3)}{\sqrt{x_1^2 + x_2^2 + x_3^2}}.$$

De esta manera, se garantiza que $|\vec{r}_i| = 1$ para toda partícula.

2. **Cálculo de la energía y de las fuerzas.** La energía electrostática total se calcula mediante la función `f_u(crd)`. Por otro lado, la fuerza resultante que siente la partícula k se calcula en la función `F_vect(k, crd)` como:

$$\vec{F}_k = \sum_{i \neq k} \frac{\vec{r}_k - \vec{r}_i}{|\vec{r}_k - \vec{r}_i|^3},$$

que corresponde a la expresión habitual de la fuerza electrostática (en unidades reducidas) debida a todas las demás partículas.

3. **Definición del parámetro de paso γ .** En cada iteración del método, el parámetro γ se calcula de forma adaptativa según:

$$\gamma = \frac{K}{F_{\text{máx}}},$$

donde $F_{\text{máx}}$ es el módulo de la fuerza más intensa entre todas las partículas en la configuración actual. Este valor se determina mediante la función `F_max(crd)`. Esta definición asegura que el desplazamiento máximo de cualquier partícula sea controlado y que el sistema no diverja numéricamente durante la evolución.

4. **Iteración Claxton–Benson.** El núcleo del método se encuentra en la función `CB(crd)`, la cual realiza w iteraciones. En cada iteración:

- Se calcula la fuerza $\vec{F}_j$ sobre cada partícula j .
- Se actualiza su posición siguiendo:

$$\vec{r}_j = \frac{\vec{r}_j + \gamma \vec{F}_j}{|\vec{r}_j + \gamma \vec{F}_j|}.$$

La normalización garantiza que la partícula permanezca sobre la esfera unitaria.

Este proceso desplaza todas las partículas de forma simultánea en la dirección de la fuerza neta que experimentan, pero con un paso reducido proporcional a $1/F_{\text{máx}}$.

5. **Criterio de convergencia y resultados.** Al finalizar, se calcula la energía total U_{obt} y se compara con el valor exacto del problema de Thomson para el mismo número de partículas. Las posiciones finales de las partículas, junto con la energía, se guardan en el archivo de texto llamado `coord_C_w.txt` (w es el número de iteraciones especificado).

Claxton-Benson y Random Walk (método 3)

Considerando N partículas, el tercer método implementado consiste en tomar la configuración generada por el método 1 (*random walk*) y aplicarle a esta el método 2 (*Claxton-Benson*). Las posiciones finales de las partículas, junto con la energía, se guardan en el archivo de texto llamado `coord_R-C_w.txt` (w es el número de iteraciones especificado). Si bien en el método 3 no se usa directamente m , es necesario debido a que de esta manera se le especifica a el programa el nombre del archivo que contiene las configuraciones generadas en el archivo 1 (El algoritmo está implementado de tal manera que sólo es necesario especificar m).

A continuación se presentan los resultados obtenidos por los tres métodos, además se enuncian algunas conclusiones.

Resultados y Conclusiones

Se muestran las energías obtenidas para cada uno de los métodos usados, mostrando buena relación con los resultados exactos. Es importante mencionar que en lo que sigue se buscaron valores óptimos de m y w , tales que su tamaño no implicara inestabilidad numérica. Se tienen las siguientes conclusiones:

- Al aumentar mucho el número de partículas N (alrededor de miles) en la simulación, el método 2 empieza a ser ineficiente, y el error producido por este es mucho mayor que el producido con el método 1.
- Para N alrededor de cientos, el método 2 produce mejores resultados.
- Para valores medianos de N , el método 3 produce más error que el método 1 y 2, es decir q mejor que el método 2 pero peor que el 1.
- A medida que N aumenta (alrededor de 1000), son necesarios menos ciclos en ambos métodos. Esto se puede entender debido a que las configuraciones para N grande son cada vez más cercanas a la del equilibrio: las fuerzas resultantes se vuelven mayores, y en el esquema de Claxton y Benson las posiciones casi no se actualizan, ya que γ en estos casos es cercano a 0. De la misma manera, para el método 1, al haber muchas partículas, es poco probable que se generen movimientos significativos en cada intento de Monte Carlo.

Las siguientes tablas muestran las energías obtenidas para los distintos números de partículas considerados, $N = 2, 3, 4, 10, 20, 40, 80, 100, 1000, 10000$, empleando los tres métodos; además se compara con los valores proporcionados en [2]. Posteriormente se indican los tiempos de cómputo correspondientes a los resultados de las tablas anteriores, además se comparan con

aquellos que toma google Colab. Por último se especifican los detalles relevantes del hardware usado, así como la configuración resultante para $N = 1000$ obtenida con el método 2.

N	Energía obtenida	Resultado exacto	Error porcentual	Calificación
2	0.5000001	0.5	2×10^{-5}	A
3	1.732051	1.732050	4×10^{-5}	A
4	3.674237	3.674234	7.1×10^{-5}	A
10	32.7204	32.7169	0.0106	A
20	150.9069	150.8815	0.0167	A
40	660.9147	660.6752	0.0362	A
80	2806.2769	2805.3558	0.0328	A
100	4450.1640	4448.3506	0.0407	A
10^3	482686.4155	482535.0	0.0313	A
10^4	49476969.8320	49447400.0	0.0590	B

Cuadro 1: Resultados obtenidos usando el método 1

N	Energía obtenida	Resultado exacto	Error porcentual	Calificación
2	0.5	0.5	0.0	A
3	1.7320508075	1.7320508075	0.0	A
4	3.67423461418	3.67423461417	1.2×10^{-14}	A
10	32.71694946	32.7169494	1.8×10^{-7}	A
20	150.88156833	150.8815683	2.2×10^{-8}	A
40	660.67527883	660.6752788	5.2×10^{-9}	A
80	2805.35587598	2805.3558759	2.8×10^{-8}	A
100	4448.350636	4448.350634	5.5×10^{-8}	A
10^3	482535.44	482535.0	9.1×10^{-5}	A
10^4	49770585.93	49447400.0	0.6535	C

Cuadro 2: Resultados obtenidos usando el método 2

N	Energía obtenida	Resultado exacto	Error porcentual	Calificación
2	0.5	0.5	0.0	A
3	1.732050807	1.732050807	0.0	A
4	3.674234614175	3.674234614174	1.2×10^{-14}	A
10	32.71694946	32.7169494	1.8×10^{-7}	A
20	150.88156833	150.8815683	2.2×10^{-8}	A
40	660.67527883	660.6752788	5.2×10^{-9}	A
80	2805.54	2805.35	.0067	A
100	4448.41	4448.35	0.0013	A
10^3	482536.044	482535.0	.00021	A
10^4	49659589.80	49447400.0	0.4291	B

Cuadro 3: Resultados obtenidos usando el método 3

N	# de iteraciones	Tiempo (s)
2	10^5	0.2146
3	10^5	0.2762
4	10^5	0.2306
10	10^5	0.3177
20	10^5	0.5110
40	10^5	1.1677
80	50×10^4	1.9196
100	50×10^4	2.7541
10^3	2.5×10^4	112.1021
10^4	500	282.4678

Cuadro 4: Método 1 en PC

N	# de iteraciones	Tiempo (s)
2	10^5	0.3348
3	10^5	0.4303
4	10^5	0.3597
10	10^5	0.4956
20	10^5	0.7972
40	10^5	1.8220
80	50×10^4	2.9949
100	50×10^4	4.2934
10^3	2.5×10^4	174.7876
10^4	500	384.1562

Cuadro 5: Método 1 en Colab

N	# de iteraciones	Tiempo (s)
2	5×10^4	0.4731
3	5×10^4	0.4226
4	5×10^4	0.4211
10	5×10^4	0.4561
20	5×10^4	0.5680
40	5×10^4	1.0421
80	5×10^3	0.6601
100	5×10^3	0.8014
10^3	10^3	8.3177
10^4	200	86.6182

Cuadro 6: Método 2 en PC

N	# de iteraciones	Tiempo (s)
2	5×10^4	0.7384
3	5×10^4	0.6593
4	5×10^4	0.6564
10	5×10^4	0.7114
20	5×10^4	0.8885
40	5×10^4	1.6278
80	5×10^3	1.0298
100	5×10^3	1.2492
10^3	10^3	12.9889
10^4	200	117.8007

Cuadro 7: Método 2 en Colab

N	# de iteraciones	Tiempo (s)
2	5×10^4	0.4359
3	5×10^4	0.4347
4	5×10^4	0.4280
10	5×10^4	0.4648
20	5×10^4	0.5765
40	5×10^4	1.1210
80	5×10^3	0.6753
100	5×10^3	0.8188
10^3	10^3	8.2322
10^4	200	87.2505

Cuadro 8: Método 3 en PC

N	# de iteraciones	Tiempo (s)
2	5×10^4	0.6797
3	5×10^4	0.6787
4	5×10^4	0.6677
10	5×10^4	0.7251
20	5×10^4	0.8995
40	5×10^4	1.7488
80	5×10^3	1.0547
100	5×10^3	1.2773
10^3	10^3	12.8462
10^4	200	118.6606

Cuadro 9: Método 3 en Colab

El **dispositivo usado** fué una Mac M1 del 2020, cuyas características son:

- cpu de 8 núcleos, arquitectura ARMv8-A de 64 bits
- 4 de los núcleos de 3.2 Ghz, los otros 4 de 2.1 Ghz
- caches asociados de 12 Mb y 4 Mb respectivamente
- ancho de banda de memoria de 68.25 GB/s
- Memoria unificada de 8 GB LPDDR4X a 4266 MHz, compartida entre CPU y GPU, con arquitectura de acceso unificado.

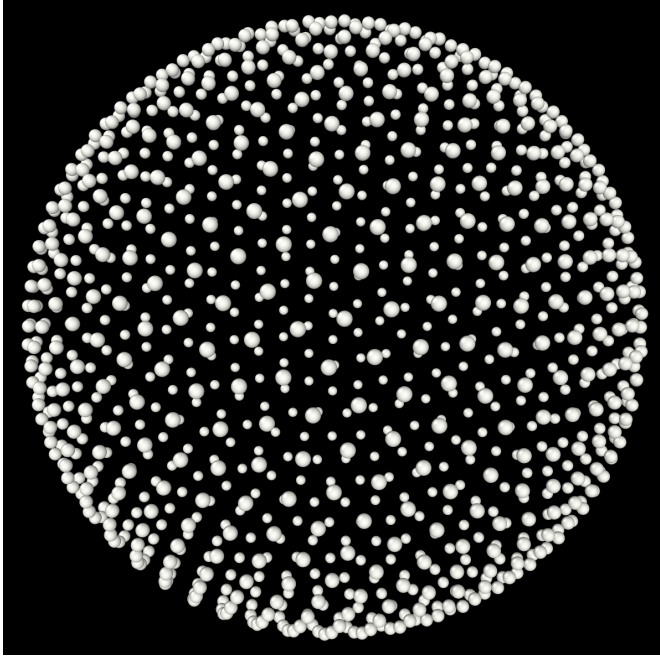


Figura 1: Configuración de $N = 1000$ partículas obtenidas usando el método de Claxton y Benson (método 2), el radio de la esfera es 1. Por motivos de visualización, se les ha asignado radio a las partículas.

Demostración

La energía electrostática del conjunto de las N partículas es

$$U_T = \frac{1}{2} \sum_{i=1}^N \sum_{\substack{j=1 \\ j \neq i}}^N \frac{1}{r_{i,j}},$$

donde $r_{i,j}$ es la distancia entre las partículas i y j . Fijándonos en una partícula arbitraria, la k -ésima, lo anterior se puede reescribir como

$$U_T = \frac{1}{2} \sum_{\substack{i=1 \\ i \neq k}}^N \sum_{\substack{j=1 \\ j \neq i}}^N \frac{1}{r_{i,j}} + \frac{1}{2} \sum_{\substack{j=1 \\ j \neq k}}^N \frac{1}{r_{k,j}}.$$

Intercambiando el orden de sumación en el primer término del lado derecho de la ecuación anterior, se obtiene

$$U_T = \frac{1}{2} \sum_{j=1}^N \sum_{\substack{i=1 \\ i \neq k \\ i \neq j}}^N \frac{1}{r_{i,j}} + \frac{1}{2} \sum_{\substack{j=1 \\ j \neq k}}^N \frac{1}{r_{k,j}}.$$

De la misma manera que al principio, fijándonos en la partícula k , podemos reescribir la energía electrostática como

$$U_T = \frac{1}{2} \sum_{\substack{j=1 \\ j \neq k}}^N \sum_{\substack{i=1 \\ i \neq k \\ i \neq j}}^N \frac{1}{r_{i,j}} + \frac{1}{2} \sum_{\substack{i=1 \\ i \neq k}}^N \frac{1}{r_{i,k}} + \frac{1}{2} \sum_{\substack{j=1 \\ j \neq k}}^N \frac{1}{r_{k,j}}.$$

Debido a que $r_{i,j}$ es simétrico en los índices, lo anterior es igual a

$$U_T = \frac{1}{2} \sum_{\substack{j=1 \\ j \neq k}}^N \sum_{\substack{i=1 \\ i \neq k \\ i \neq j}}^N \frac{1}{r_{i,j}} + \sum_{\substack{i=1 \\ i \neq k}}^N \frac{1}{r_{i,k}}.$$

Nótese que no existe dependencia en el primer término del lado derecho de la ecuación respecto de la posición de la partícula k . Si $r'_{i,k}$ representa la distancia entre la partícula i y la partícula k al haber cambiado esta última de posición, entonces se obtiene una nueva energía:

$$U'_T = \frac{1}{2} \sum_{\substack{j=1 \\ j \neq k}}^N \sum_{\substack{i=1 \\ i \neq k \\ i \neq j}}^N \frac{1}{r_{i,j}} + \sum_{\substack{i=1 \\ i \neq k}}^N \frac{1}{r'_{i,k}}.$$

Si se restan ambas ecuaciones se obtiene

$$U'_T - U_T = \sum_{\substack{i=1 \\ i \neq k}}^N \left(\frac{1}{r'_{i,k}} - \frac{1}{r_{i,k}} \right) = \Delta U,$$

de manera que

$$U'_T = \Delta U + U_T.$$

Referencias

- [1] O. Steinhauser, *Computer Simulation in Physics and Engineering*, De Gruyter, 2013.
- [2] D. J. Wales y S. Ulker, “Global Minima for the Thomson Problem,” *The Cambridge Cluster Database*, disponible en: <https://www-wales.ch.cam.ac.uk/~wales/CCD/Thomson/table.html>